

Université Hassan I
École Nationale des Sciences Appliquées – Berrechid

Rapport technique :

Sous le thème :

Data-Dive : Exploration Collaborative et Asymétrique de Graphes Massifs



Réalisé par :

Amar Chaimaa
Assmaa Azaroual
Bensaid Malak
Basma El mghari
Mohamed Smaoui

Encadré par :

Pr. Hrimech
Pr. Oumaima

Année universitaire : 2025 – 2026

Remerciements

Avant de présenter ce travail, nous tenons à exprimer notre profonde gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Nous adressons tout particulièrement nos sincères remerciements à **M. Hrimech**, notre encadrant pédagogique et enseignant du module, pour son accompagnement, sa disponibilité, ses précieux conseils ainsi que son implication tout au long du projet. Son encadrement nous a permis de mener ce travail dans les meilleures conditions.

Nous remercions également **M.Oumaima** pour le temps qu'il nous a consacré, pour son encadrement et pour les connaissances qu'il nous a transmises dans le domaine des technologies de la réalité virtuelle. Son expertise sur les environnements VR et l'utilisation des casques de réalité virtuelle ont constitué une aide précieuse pour la réussite de ce projet.

Nos remerciements s'adressent également à notre établissement **ENSA Berrechid**, qui a mis à notre disposition les ressources matérielles, les équipements ainsi que l'environnement de travail nécessaires au développement et à la réalisation de ce projet.

Enfin, nous souhaitons remercier chacun des membres de notre groupe pour leur engagement, leur investissement personnel, leur esprit de collaboration et le temps consacré à la réalisation de ce travail. Grâce aux efforts collectifs de tous les participants, nous avons pu mener à bien ce projet et atteindre les objectifs fixés. À toutes ces personnes, nous adressons nos plus sincères remerciements.

Résumé

Ce rapport présente le projet *Data-Dive : Exploration Collaborative et Asymétrique de Graphes Massifs*, développé dans le cadre du thème *Immersive Analytics et Data-Diving sur des Graphes Complexes*. L'objectif principal du projet est de transformer un jeu massif de données complexes, notamment des transactions financières destinées à la détection de fraude, en un univers immersif tridimensionnel exploitable dans un environnement de réalité virtuelle.

Le projet repose sur une approche combinant **Big Data**, **Intelligence Artificielle** et **visualisation immersive**. Les données sont d'abord traitées à grande échelle à l'aide de technologies adaptées aux graphes complexes et aux volumes massifs de données. Ensuite, des algorithmes de réduction de dimensionnalité, tels que *PCA*, *UMAP* et *t-SNE*, permettent de projeter les données dans un espace tridimensionnel cohérent, tandis que des techniques de clustering regroupent automatiquement les nœuds présentant des comportements similaires. En parallèle, des méthodes de détection d'anomalies basées sur l'intelligence artificielle permettent d'identifier et de mettre en évidence les comportements suspects ou frauduleux.

L'originalité du projet réside également dans son interaction collaborative asymétrique. D'un côté, l'utilisateur en réalité virtuelle explore directement les données sous forme de clusters manipulables dans un environnement immersif. De l'autre côté, un utilisateur desktop agit comme analyste, exécute des requêtes avancées et modifie dynamiquement la structure de visualisation observée par le joueur VR. Cette approche vise à améliorer la compréhension de graphes massifs complexes grâce à une interaction plus intuitive, collaborative et visuelle.

Enfin, ce travail ouvre des perspectives scientifiques concernant l'évaluation de l'efficacité des environnements immersifs pour la détection d'anomalies par rapport aux interfaces classiques en deux dimensions, ainsi que l'étude ergonomique des interactions collaboratives asymétriques dans l'analyse de données complexes.

Table des matières

Remerciements	2
0.1 Introduction générale	3
0.2 Objectifs du projet	3
0.3 Architecture générale du projet Data-Dive	4
0.4 Partie 1 – Data Engineering et construction du graphe	4
0.4.1 Rôle de la Partie 1	4
0.4.2 Chargement des données	5
0.4.3 Nettoyage des transactions	5
0.4.4 Construction des nœuds et des arêtes	5
0.4.5 Calcul des caractéristiques par nœud	5
0.4.6 Résultats de la Partie 1	6
0.5 Partie 2 – Intelligence artificielle et analytics	6
0.5.1 Rôle de la Partie 2	6
0.5.2 Prétraitement et normalisation	7
0.5.3 Réduction dimensionnelle en 3D	7
0.5.4 Clustering comportemental	7
0.5.5 Détection d'anomalies	8
0.5.6 Fusion et export des résultats	8
0.5.7 Résultats de la Partie 2	8
0.6 Lien avec la visualisation immersive	9
0.7 Conclusion	9
0.8 Partie 3 – Backend API et synchronisation	10
0.8.1 Introduction	10
0.8.2 Objectifs du backend	10
0.8.3 Architecture du module	10
0.8.4 Étape 1 – Configuration et initialisation de l'API	11
0.8.5 Étape 2 – Intégration des données analytiques	11
0.8.6 Étape 3 – Définition des modèles de données	12

0.8.7	Étape 4 – Implémentation des services	12
0.8.8	Étape 5 – Exposition des ressources par API REST	12
0.8.9	Étape 6 – Synchronisation entre Desktop et Unity	13
0.8.10	Étape 7 – Validation et journalisation	13
0.8.11	Conclusion	13
0.9	Partie 4 : Module de visualisation 3D des graphes sous Unity	14
0.9.1	Introduction du module	14
0.9.2	Choix techniques et environnement de développement	14
0.9.3	Optimisation GPU par rendu indirect	14
0.9.4	Consommation des données depuis l'API	15
0.9.5	Encodage visuel et représentation algorithmique	15
0.9.6	Rendu des liens entre les nœuds	16
0.9.7	Apport du module	16
0.10	Partie 5 : Développement de l'application Web de supervision et d'analyse	16
0.10.1	Rôle de la Partie 5	16
0.10.2	Fonctionnalités développées	17
0.10.3	Interface utilisateur de l'application Web	19
0.11	Perspectives d'amélioration	20
0.11.1	Amélioration de l'environnement VR	20
0.11.2	Amélioration de la communication et de l'architecture	21
0.11.3	Travail collaboratif	21

0.1 Introduction générale

Le sujet choisi est le **Projet 4 : Immersive Analytics et Data-Diving sur des Graphes Complexes**. Le concept consiste à représenter un jeu de données massif sous la forme d'un univers 3D. Dans ce projet, les entités du jeu de données deviennent des nœuds, les relations deviennent des arêtes, et les comportements suspects sont mis en évidence visuellement.

Le cas d'usage retenu est l'analyse de transactions financières simulées à partir du dataset PaySim. Ce dataset contient des opérations financières mobiles et permet de travailler sur un problème réaliste : la détection de fraude. Le but n'est pas seulement de détecter automatiquement les anomalies, mais aussi de préparer les données pour une exploration immersive dans laquelle un utilisateur VR peut naviguer dans les clusters pendant qu'un utilisateur desktop pilote les requêtes et l'analyse.

Les deux parties décrites dans ce rapport constituent la base Big Data et IA du projet :

- **Partie 1 – Data Engineering** : chargement, nettoyage, transformation des transactions et construction du graphe.
- **Partie 2 – IA / Analytics** : normalisation des variables, projection 3D, clustering et détection d'anomalies.

0.2 Objectifs du projet

L'objectif général est de produire une chaîne de traitement capable de transformer des données financières brutes en données enrichies, structurées et prêtes pour une visualisation 3D dans Unity. Cette chaîne doit respecter plusieurs contraintes :

- gérer un volume important de nœuds et d'arêtes ;
- produire un graphe clair à partir de transactions financières ;
- calculer des caractéristiques comportementales pour chaque compte ;
- réduire les données en coordonnées 3D exploitables dans un moteur temps réel ;
- identifier des groupes de comportements similaires ;
- mettre en évidence les comptes suspects ou critiques ;
- exporter les résultats dans des formats simples à intégrer, notamment CSV, Parquet et JSON.

0.3 Architecture générale du projet Data-Dive

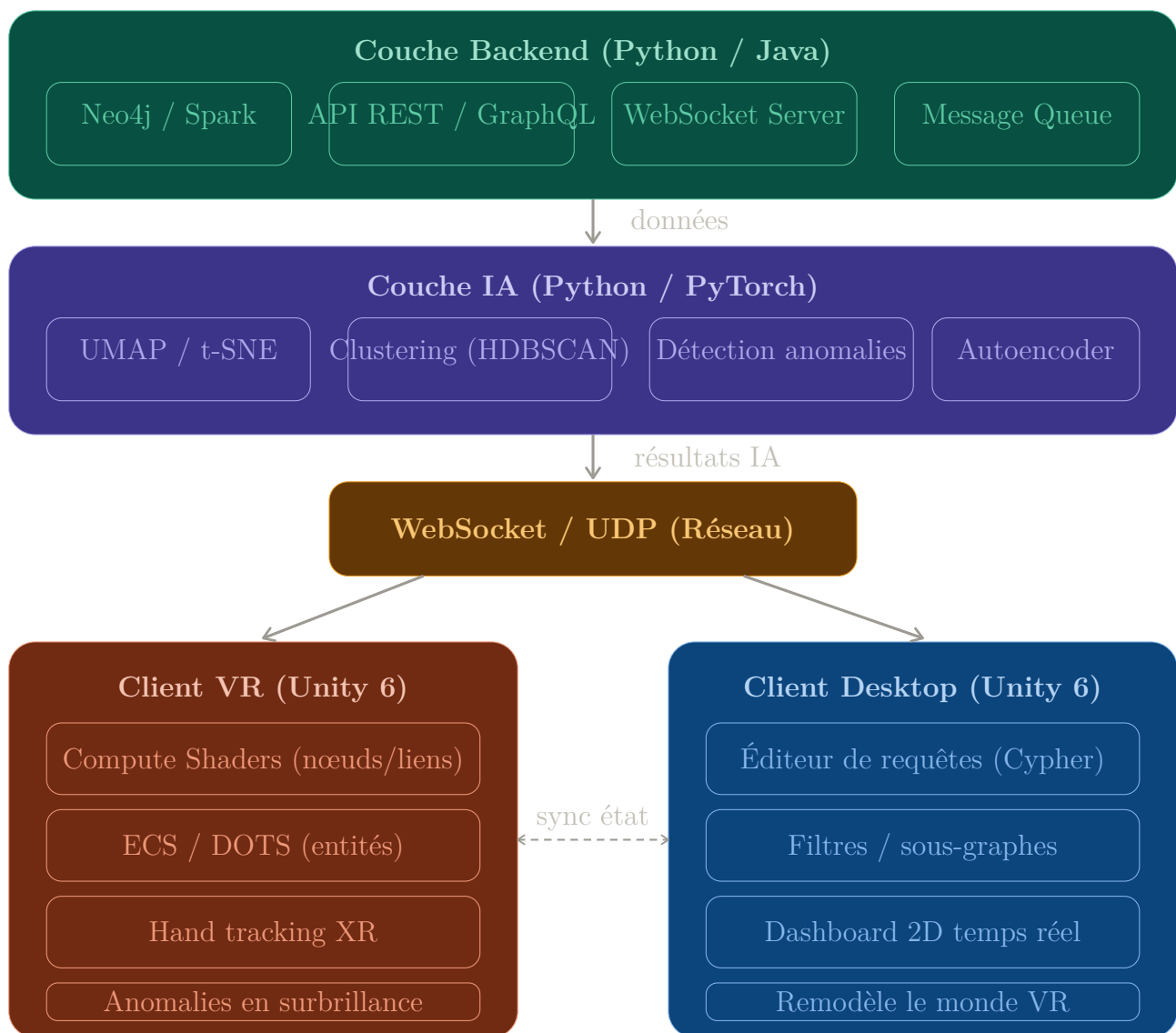


FIGURE 1 – Architecture générale du projet Data-Dive

0.4 Partie 1 – Data Engineering et construction du graphe

0.4.1 Rôle de la Partie 1

La première partie du projet prépare les données brutes pour l'analyse. Elle prend en entrée le fichier `paysim.csv`, nettoie les transactions, extrait les entités importantes, construit un graphe orienté et calcule des variables numériques par nœud.

Le pipeline réalisé est le suivant :

```
paysim.csv → load_data.py → clean_data.py → build_graph.py →  
compute_features.py → export_data.py
```

0.4.2 Chargement des données

Le fichier source utilisé est `paysim.csv`. Il s'agit d'un jeu de données simulant des transactions financières mobiles. Chaque ligne représente une transaction avec un émetteur, un destinataire, un montant, un type d'opération, des soldes avant et après transaction, ainsi que des indicateurs de fraude.

Dans le projet, les données brutes sont placées dans :

```
phase1_data_engineering/data/raw/paysim.csv
```

Le chargement est isolé dans un module spécifique afin de garder le pipeline lisible et réutilisable.

0.4.3 Nettoyage des transactions

Le nettoyage est réalisé dans le fichier `clean_data.py`. Les opérations principales sont :

- suppression des doublons ;
- suppression des montants invalides ou négatifs ;
- suppression des lignes contenant des valeurs manquantes ;
- filtrage sur les types de transactions `TRANSFER` et `CASH_OUT`, car ce sont les types les plus pertinents pour l'étude de fraude ;
- renommage des colonnes pour obtenir des noms plus explicites, par exemple `nameOrig` devient `sender` et `nameDest` devient `receiver` ;
- ajout de colonnes dérivées comme la différence de solde, le logarithme du montant et l'indicateur `orig_account_emptied`.

Ces transformations rendent les données plus propres et plus adaptées à une modélisation sous forme de graphe.

0.4.4 Construction des nœuds et des arêtes

La construction du graphe est réalisée dans `build_graph.py`. Le graphe est orienté : chaque transaction crée une arête allant du compte émetteur vers le compte destinataire.

- Un **nœud** représente un compte financier.
- Une **arête** représente une transaction entre deux comptes.
- Les arêtes conservent des informations comme le montant, le type de transaction, l'étape temporelle et l'indicateur de fraude.

Le projet utilise aussi `NetworkX` pour représenter le graphe en mémoire. Cette structure facilite l'analyse et prépare une future exploitation par un backend ou un moteur de visualisation.

0.4.5 Calcul des caractéristiques par nœud

Le fichier `compute_features.py` calcule les caractéristiques comportementales de chaque compte. Ces features décrivent le comportement d'envoi, de réception et de solde.

Exemples de caractéristiques calculées :

- nombre de transactions envoyées ;
- montant total envoyé ;
- montant moyen envoyé ;
- montant maximal envoyé ;
- nombre de transactions frauduleuses envoyées ;
- taux de fraude à l'envoi ;
- nombre de destinataires uniques ;
- nombre de transactions reçues ;
- nombre d'émetteurs uniques ;
- différence moyenne de solde ;
- ratio entre transactions envoyées et reçues ;
- volume total d'activité du compte.

Ces variables sont essentielles pour la Partie 2, car elles constituent l'entrée des algorithmes de réduction dimensionnelle, de clustering et de détection d'anomalies.

0.4.6 Résultats de la Partie 1

La Partie 1 génère plusieurs fichiers intermédiaires et finaux :

- `cleaned_transactions.csv` : transactions nettoyées ;
- `nodes.csv` : liste des comptes ;
- `edges.csv` : liste des transactions sous forme d'arêtes ;
- `nodes_features.parquet` : caractéristiques calculées par nœud ;
- `nodes.json` et `edges.json` : exports destinés à l'intégration.

D'après le fichier de synthèse généré par le projet, les résultats obtenus sont :

Indicateur	Valeur
Nombre de nœuds	244 560
Nombre d'arêtes	135 191
Nombre de features par nœud	31
Nombre de nœuds liés à la fraude	16 351
Nombre d'arêtes frauduleuses	8 197
Type de compte dominant	client

TABLE 1 – Résultats principaux de la Partie 1

0.5 Partie 2 – Intelligence artificielle et analytics

0.5.1 Rôle de la Partie 2

La deuxième partie reçoit les caractéristiques produites par la Partie 1 et applique des méthodes d'analyse automatique. Son rôle est de transformer les données tabulaires en informations directement exploitables pour la visualisation immersive.

Le pipeline réalisé est le suivant :

```
nodes_features.parquet + edges.csv → preprocess.py → reduce_dimension.py →  
clustering.py → anomaly_detection.py → merge_results.py → export_results.py
```

0.5.2 Prétraitement et normalisation

Avant l'application des modèles, les features numériques sont sélectionnées puis normalisées avec **RobustScaler**. Ce choix est adapté aux données financières, car elles contiennent souvent des valeurs extrêmes. Contrairement à une normalisation classique très sensible aux outliers, **RobustScaler** utilise des statistiques robustes comme la médiane et l'écart interquartile.

Certaines colonnes sont exclues de l'entraînement, notamment les identifiants, les types de comptes et les labels directs de fraude. Cela évite de fournir au modèle des informations qui biaiserait l'analyse comportementale.

0.5.3 Réduction dimensionnelle en 3D

La projection 3D est réalisée dans `reduce_dimension.py`. Le but est de convertir les features numériques de chaque nœud en coordonnées **x**, **y**, **z**. Ces coordonnées seront utilisées pour placer les nœuds dans l'univers 3D de Unity.

La stratégie utilisée est optimisée pour les grands volumes :

- réduction initiale par PCA vers 20 dimensions ;
- application de UMAP en 3D ;
- traitement par lots pour limiter la consommation mémoire ;
- normalisation finale des coordonnées dans l'intervalle $[-25, 25]$, pratique pour un espace Unity.

Cette étape est importante pour l'Immersive Analytics : les comptes ayant des comportements proches doivent être placés près les uns des autres, afin que l'utilisateur VR puisse percevoir visuellement des zones, des groupes et des anomalies.

0.5.4 Clustering comportemental

Le clustering est effectué dans `clustering.py`. Le projet utilise **MiniBatchKMeans** avec **k = 8**. Ce choix permet de traiter efficacement un grand nombre de nœuds en réduisant la charge mémoire et le temps de calcul par rapport à un KMeans classique.

Chaque nœud reçoit :

- un label de cluster ;
- une probabilité de cluster ;
- des statistiques de groupe, comme la taille du cluster, le taux de fraude et le montant moyen.

Ces clusters permettent de regrouper les comptes ayant des comportements proches. Dans l'application immersive, ils peuvent être représentés par des couleurs, des zones ou des formes distinctes.

0.5.5 Détection d'anomalies

La détection d'anomalies est réalisée dans `anomaly_detection.py` avec `Isolation Forest`. Cet algorithme est adapté à la détection d'observations rares ou atypiques dans des données multidimensionnelles.

Les paramètres principaux utilisés sont :

- `n_estimators = 100`;
- `contamination = 0.05`;
- apprentissage sur un échantillon si le dataset est trop grand;
- scoring par batch de 50 000 nœuds pour limiter l'utilisation mémoire.

Le score obtenu est normalisé entre 0 et 1. Plus le score est proche de 1, plus le nœud est considéré comme anormal. Le projet classe ensuite les nœuds en trois niveaux de risque :

- **normal**;
- **suspect**;
- **critique**.

0.5.6 Fusion et export des résultats

Après la projection 3D, le clustering et la détection d'anomalies, tous les résultats sont fusionnés dans un fichier final. Ce fichier contient à la fois les informations initiales du nœud, ses coordonnées 3D, son cluster, son score d'anomalie et les attributs utiles pour l'affichage.

Les exports principaux sont :

- `nodes_3d.parquet` : coordonnées 3D des nœuds;
- `clusters.parquet` : labels et statistiques de clusters;
- `anomalies.parquet` : scores et niveaux de risque;
- `final_nodes.parquet` : fichier final fusionné;
- `nodes_3d.json`, `clusters.json`, `anomalies.json` : exports JSON destinés à Unity ou au backend.

0.5.7 Résultats de la Partie 2

D'après le fichier `summary_phase2.json`, la Partie 2 a produit les résultats suivants :

Indicateur	Valeur
Nombre de nœuds analysés	244 560
Nombre de clusters	8
Nombre d'anomalies détectées	12 250
Nœuds classés normaux	226 959
Nœuds classés suspects	13 586
Nœuds classés critiques	4 015

TABLE 2 – Résultats principaux de la Partie 2

Le cluster le plus volumineux contient 177 665 nœuds, tandis que certains clusters plus petits présentent des taux de fraude ou des scores d'anomalie plus élevés. Par exemple, le cluster 6 contient 1 024 nœuds et présente un score moyen d'anomalie élevé. Le cluster 4 contient 6 343 nœuds et concentre un nombre important de nœuds frauduleux. Ces observations montrent l'intérêt d'une exploration visuelle : les zones rares ou denses peuvent guider l'utilisateur vers des comportements suspects.

0.6 Lien avec la visualisation immersive

Les résultats des Parties 1 et 2 préparent directement la scène immersive :

- les nœuds deviennent des objets 3D dans Unity ;
- les arêtes représentent les transactions entre comptes ;
- les coordonnées UMAP définissent la position des nœuds dans l'espace ;
- les clusters peuvent être représentés par des couleurs ;
- les anomalies peuvent être mises en évidence par une taille, une couleur ou une forme différente ;
- les niveaux **normal**, **suspect** et **critique** facilitent l'interprétation rapide par l'utilisateur.

Cette préparation permet au joueur VR de plonger dans les données, d'observer les groupes de comptes et d'identifier visuellement les régions suspectes. De son côté, l'utilisateur desktop peut jouer le rôle d'analyste en filtrant, interrogeant ou reconfigurant la visualisation.

Lien de la vidéo : *à compléter*

0.7 Conclusion

Les Parties 1 et 2 du projet *Data-Dive* ont permis de transformer un dataset financier brut en un graphe enrichi et analysé par intelligence artificielle. La Partie 1 a produit une base structurée composée de nœuds, d'arêtes et de features. La Partie 2 a ajouté une couche analytique avec une projection 3D, des clusters et des scores d'anomalie.

Ces résultats constituent une base solide pour la suite du projet : l'intégration dans un environnement Unity immersif, avec une interaction asymétrique entre un utilisateur VR et un utilisateur desktop. Le travail réalisé montre comment la Big Data, l'IA et la réalité virtuelle

peuvent être combinées pour rendre l’analyse de graphes massifs plus intuitive, collaborative et visuelle.

0.8 Partie 3 – Backend API et synchronisation

0.8.1 Introduction

La troisième partie du projet *Data-Dive* concerne la mise en place du backend applicatif. Elle occupe une position centrale dans l’architecture générale, car elle assure la communication entre les traitements analytiques réalisés en amont et les interfaces de visualisation développées en aval.

La Partie 2 produit un graphe enrichi comprenant les coordonnées tridimensionnelles des nœuds, les arêtes transactionnelles, les clusters comportementaux et les anomalies détectées. Le backend récupère ces résultats, les organise et les expose sous la forme d’une API. Les applications Unity VR et desktop peuvent ainsi accéder aux mêmes données et partager un point de synchronisation commun.

Partie 2 : IA et Analytics → Partie 3 : Backend API → Parties 4 et 5 : Unity
VR et Desktop

0.8.2 Objectifs du backend

Le backend remplit trois fonctions principales :

- exposer les données du graphe à travers une interface HTTP structurée ;
- fournir des ressources dédiées aux nœuds, aux arêtes, aux clusters et aux anomalies ;
- préparer la synchronisation entre l’utilisateur desktop et l’utilisateur VR dans le cadre de l’exploration collaborative asymétrique.

Cette couche intermédiaire permet de découpler les traitements de données des applications clientes. Unity et l’interface desktop n’ont donc pas besoin de connaître l’organisation interne des pipelines d’analyse : ils communiquent uniquement avec l’API.

0.8.3 Architecture du module

Le backend est organisé dans le répertoire `phase3_backend_api`. Sa structure principale est la suivante :

```
phase3_backend_api/  
--> README.md  
--> requirements.txt  
--> .env  
--> app/  
--> -->main.py
```

```
--> -->config.py
--> -->routes/
--> -->services/
--> -->models/
-->data/
-->tests/
-->logs/
```

Cette organisation sépare les responsabilités techniques du backend. Le répertoire **routes** définit les points d'accès de l'API, le répertoire **services** contient la logique métier et le répertoire **models** décrit les structures de données manipulées. Le dossier **data** regroupe les fichiers utilisés par l'application, tandis que les dossiers **tests** et **logs** sont respectivement consacrés à la validation et à la traçabilité.

0.8.4 Étape 1 – Configuration et initialisation de l'API

Le backend est développé en Python avec le framework **FastAPI**. Ce choix est adapté à la création d'API légères et modulaires. FastAPI facilite également la définition des routes, la validation des paramètres et la génération automatique d'une documentation interactive.

Le fichier `app/main.py` constitue le point d'entrée de l'application.

Le fichier `app/config.py` centralise les paramètres configurables, notamment le nom de l'application, l'environnement d'exécution, le préfixe des routes, les origines autorisées et les chemins d'accès aux données. Le fichier `.env` permet de modifier ces paramètres sans altérer directement le code source.

Deux routes générales facilitent la supervision du serveur :

- `/` retourne un état synthétique de l'application ;
- `/health` vérifie que le service fonctionne et que le répertoire de données est accessible.

0.8.5 Étape 2 – Intégration des données analytiques

Le backend exploite les exports produits par les phases précédentes. Les fichiers attendus dans le répertoire **data** sont :

- `nodes_3d.json` : nœuds enrichis avec leurs coordonnées tridimensionnelles ;
- `edges.json` : relations transactionnelles entre les comptes ;
- `clusters.json` : informations relatives aux groupes comportementaux ;
- `anomalies.json` : scores d'anomalie et niveaux de risque.

Ces fichiers constituent une représentation directement exploitable du graphe intelligent produit par la Partie 2. Le backend charge les données nécessaires et peut les conserver temporairement en mémoire afin de limiter les lectures répétées. Cette stratégie est adaptée à un prototype local et facilite les échanges avec les applications clientes.

0.8.6 Étape 3 – Définition des modèles de données

Les structures manipulées par l'API sont décrites dans le répertoire `app/models`. Chaque ressource possède un modèle dédié :

- `node_model.py` décrit les nœuds, leurs positions 3D, leurs clusters et leurs niveaux de risque ;
- `edge_model.py` décrit les arêtes, leurs extrémités et les informations transactionnelles associées ;
- `cluster_model.py` décrit les groupes de nœuds et leurs statistiques principales ;
- `anomaly_model.py` décrit les anomalies, leur score et leur niveau de criticité.

Cette modélisation améliore la lisibilité du projet et définit un contrat de données commun entre le backend, Unity et l'application desktop.

0.8.7 Étape 4 – Implémentation des services

La logique métier est regroupée dans le répertoire `app/services`. Cette couche traite les demandes reçues par l'API avant de retourner les résultats aux clients.

- `graph_service.py` gère la consultation des nœuds et des arêtes ;
- `cluster_service.py` fournit les informations liées aux clusters ;
- `anomaly_service.py` organise les résultats de détection d'anomalies ;
- `sync_service.py` gère l'état des données et la diffusion des événements de synchronisation.

Cette séparation évite de placer la logique de traitement directement dans les routes HTTP. Elle rend l'application plus facile à maintenir, à tester et à faire évoluer.

0.8.8 Étape 5 – Exposition des ressources par API REST

Les points d'accès HTTP sont définis dans le répertoire `app/routes`. Ils permettent aux clients de consulter les différentes composantes du graphe enrichi.

Méthode	Route	Fonction principale
GET	<code>/api/nodes</code>	Consulter et filtrer les nœuds
GET	<code>/api/nodes/summary</code>	Obtenir une synthèse des nœuds
GET	<code>/api/nodes/{node_id}</code>	Consulter un nœud précis
GET	<code>/api/edges</code>	Consulter et filtrer les arêtes
GET	<code>/api/clusters</code>	Consulter les clusters
GET	<code>/api/clusters/{label}/nodes</code>	Consulter les nœuds d'un cluster
GET	<code>/api/anomalies</code>	Consulter et filtrer les anomalies
GET	<code>/api/anomalies/top</code>	Obtenir les anomalies prioritaires
GET	<code>/api/anomalies/summary</code>	Obtenir une synthèse des anomalies

TABLE 3 – Principales routes REST du backend

La pagination repose sur les paramètres `limit` et `offset`. Elle permet de limiter le nombre

d'éléments transmis au client lorsque le graphe contient un volume important de données. Des filtres complètent ce mécanisme : les nœuds peuvent être sélectionnés selon leur cluster ou leur niveau de risque, tandis que les arêtes peuvent être filtrées selon leur source, leur destination, leur type ou leur caractère frauduleux.

0.8.9 Étape 6 – Synchronisation entre Desktop et Unity

Le fichier `sync.py` définit les routes consacrées à la synchronisation. Cette fonctionnalité est essentielle pour l'analyse collaborative asymétrique : l'utilisateur desktop doit pouvoir effectuer des actions susceptibles d'influencer la scène observée par l'utilisateur VR.

Les routes principales sont :

- `/api/sync/status` pour consulter l'état des données disponibles ;
- `/api/sync/import` pour importer les exports analytiques ;
- `/api/sync/reload` pour recharger les données ;
- `/api/sync/ws` pour établir une communication WebSocket.

Le canal WebSocket permet de diffuser des événements aux clients connectés. Cette base technique peut être utilisée pour transmettre des changements de filtre, des sélections de nœuds, des demandes de rechargement ou des actions d'analyse entre l'interface desktop et l'environnement Unity.

0.8.10 Étape 7 – Validation et journalisation

Le répertoire `tests` contient des tests automatisés consacrés aux comportements principaux de l'API. Les fichiers `test_nodes.py` et `test_anomalies.py` vérifient notamment la consultation des nœuds, le filtrage des arêtes frauduleuses, le classement des anomalies et l'état du service de synchronisation.

Le répertoire `logs` contient le fichier `api.log`. Celui-ci conserve les informations d'exécution utiles au suivi de l'application et facilite l'identification d'éventuelles erreurs lors du chargement des données ou du traitement des requêtes.

0.8.11 Conclusion

La Partie 3 assure la transition entre l'analyse des données et leur exploitation interactive. Grâce à FastAPI, le projet dispose d'une interface commune pour accéder aux nœuds, aux arêtes, aux clusters et aux anomalies. Le mécanisme de synchronisation prépare également la collaboration entre l'utilisateur desktop et l'utilisateur VR. Le backend constitue ainsi le point central de communication de l'architecture *Data-Dive*.

0.9 Partie 4 : Module de visualisation 3D des graphes sous Unity

0.9.1 Introduction du module

Dans le cadre du projet de détection de fraudes et d'analyse de réseaux, un module de visualisation 3D a été développé afin de représenter de manière immersive un graphe de grande taille. Ce graphe est composé de 3993 nœuds et de plusieurs milliers de liens, récupérés depuis une API REST locale connectée à une base de données de graphes. L'objectif principal de ce module est de permettre une exploration visuelle fluide du réseau, tout en mettant en évidence les nœuds présentant un comportement frauduleux ou un niveau de risque critique.

La visualisation de graphes constitue un enjeu important dans l'analyse de fraudes, car elle permet d'observer les relations entre entités, de détecter des regroupements suspects et d'identifier rapidement les acteurs centraux ou anormaux du réseau. Le choix d'une représentation tridimensionnelle vise à améliorer la lisibilité lorsque le nombre de nœuds et de liens devient élevé, en exploitant l'espace 3D pour réduire la superposition visuelle et faciliter l'interprétation.

0.9.2 Choix techniques et environnement de développement

Le moteur de visualisation a été réalisé avec Unity 6, version 6000.4, en utilisant le pipeline de rendu universel URP, ou *Universal Render Pipeline*. Ce choix permet de bénéficier d'un pipeline moderne, compatible avec des rendus optimisés et adapté aux applications interactives en temps réel. Unity a également été retenu pour sa capacité à produire des environnements immersifs, notamment en réalité virtuelle, ce qui constitue une perspective importante pour l'exploration avancée de graphes complexes.

Cependant, l'affichage direct de milliers de nœuds sous forme de *GameObjects* classiques, par exemple des sphères Unity instanciées individuellement, pose un problème majeur de performance. Chaque objet ajoute une charge de calcul côté processeur, notamment pour la gestion des transformations, du rendu, des composants et des appels de dessin. Dans un graphe de plusieurs milliers de nœuds, cette approche provoquerait une chute importante du taux de rafraîchissement, rendant l'expérience difficilement exploitable, en particulier dans un contexte immersif où la fluidité est essentielle.

0.9.3 Optimisation GPU par rendu indirect

Afin de répondre à cette contrainte, le module utilise une stratégie de rendu indirect à haute performance basée sur la méthode native `Graphics.DrawMeshInstancedIndirect`. Contrairement à une instanciation classique de *GameObjects*, cette approche délègue une grande partie du travail de rendu à la carte graphique. Les nœuds du graphe ne sont donc pas créés comme des objets Unity indépendants, mais sont représentés sous forme d'instances graphiques envoyées directement au GPU.

Les informations nécessaires à l'affichage des 3993 nœuds, notamment leurs positions, leurs échelles et leurs couleurs, sont stockées dans un `GraphicsBuffer`. Ce buffer structuré est ensuite transmis à la carte graphique, qui l'utilise pour dessiner l'ensemble des instances en un nombre

réduit d'appels de rendu. Cette architecture réduit fortement la surcharge côté CPU et améliore la stabilité du nombre d'images par seconde.

Un shader HLSL personnalisé, nommé `NodeURPShader`, a été développé pour être compatible avec le pipeline URP. Son rôle est d'interpréter les données contenues dans le buffer d'instances, puis de projeter chaque nœud à l'écran avec sa position, sa taille et sa couleur propres. Cette solution rend possible l'affichage simultané de milliers de sphères tout en conservant une expérience interactive fluide. Elle constitue donc un choix technique essentiel pour assurer la viabilité du module dans des environnements exigeants comme la réalité virtuelle.

0.9.4 Consommation des données depuis l'API

Le module Unity communique avec une API REST locale afin de récupérer dynamiquement les données du graphe. Les routes `/api/nodes` et `/api/edges` sont consommées de manière asynchrone à l'aide de `UnityWebRequest` et de Coroutines Unity. Cette approche permet de charger les données sans bloquer le fil d'exécution principal de l'application, ce qui préserve la fluidité de l'interface pendant les opérations réseau.

Après récupération des données, une étape de filtrage est appliquée afin de ne conserver que les nœuds réellement reliés à des liens valides. Ce traitement évite de charger inutilement des entités isolées ou incohérentes, et garantit une correspondance correcte entre les sources et les cibles des arêtes. Le graphe affiché correspond ainsi à une structure exploitable et cohérente avec les données fournies par l'API.

0.9.5 Encodage visuel et représentation algorithmique

La visualisation ne se limite pas à afficher les nœuds et les liens ; elle encode également des informations analytiques directement dans la scène 3D. Les coordonnées des nœuds sont automatiquement mises à l'échelle et dispersées selon les axes X, Y et Z afin d'aérer le graphe dans l'espace. Cette transformation améliore la lisibilité globale et limite la concentration excessive des nœuds dans une zone restreinte.

Les nœuds normaux sont colorés selon leur communauté ou groupe d'appartenance, à partir de la variable `cluster_label`. Cette coloration par cluster permet d'identifier visuellement les sous-structures du réseau et de repérer les groupes fortement connectés. Les couleurs utilisées, telles que le bleu, le vert ou le blanc, facilitent la distinction entre communautés sans perturber la lecture globale du graphe.

Les nœuds frauduleux ou critiques sont traités de manière spécifique. Lorsqu'un nœud possède l'attribut `is_fraud_node` égal à 1, il est automatiquement agrandi et coloré en rouge vif. Sa taille est doublée par rapport aux nœuds standards afin de le rendre immédiatement visible dans la scène. Ce choix de *data mapping* permet de hiérarchiser l'information visuelle et d'attirer l'attention de l'utilisateur sur les entités les plus importantes pour l'analyse de fraude.

0.9.6 Rendu des liens entre les nœuds

Pour représenter les relations entre les entités du graphe, une architecture hybride a été adoptée. Les nœuds sont rendus de manière optimisée par le GPU grâce au rendu indirect, tandis que les liens sont tracés dynamiquement à l'aide de composants Unity `LineRenderer`. Chaque lien relie directement les coordonnées 3D du nœud source et du nœud cible issues des données de l'API.

Cette approche hybride présente un compromis pertinent entre performance et stabilité visuelle. Le rendu indirect est utilisé pour la partie la plus coûteuse, à savoir l'affichage massif des nœuds, tandis que les `LineRenderer` offrent une solution simple et fiable pour matérialiser les connexions. Les arêtes restent ainsi lisibles, correctement positionnées et cohérentes avec la structure réelle du graphe.

0.9.7 Apport du module

Le module développé apporte une contribution importante au projet global, car il transforme les résultats de l'analyse de graphes en une représentation visuelle interactive et exploitable. Grâce au rendu indirect par instancing GPU, le système dépasse les limites d'une approche classique basée sur des milliers de *GameObjects*. Cette optimisation rend possible l'affichage fluide d'un graphe volumineux et prépare le projet à des usages immersifs, notamment en réalité virtuelle.

En combinant performance graphique, consommation dynamique des données d'API et encodage visuel des risques, ce module constitue un outil d'aide à l'analyse. Il permet à l'utilisateur d'explorer les communautés du réseau, d'observer les connexions entre entités et d'identifier rapidement les nœuds frauduleux ou critiques.

0.10 Partie 5 : Développement de l'application Web de supervision et d'analyse

0.10.1 Rôle de la Partie 5

Cette phase du projet a consisté à développer une application Web interactive permettant à l'analyste d'explorer et d'analyser les données dans un environnement 2D complémentaire à l'expérience immersive proposée en réalité virtuelle.

L'objectif principal est de fournir une interface de supervision facilitant l'analyse des données, la détection d'anomalies et l'interaction avec l'environnement VR.

0.10.2 Fonctionnalités développées

Visualisation 2D des données

Une interface Web a été développée afin d'offrir une représentation graphique des données sous forme de tableaux de bord interactifs. Cette vue permet aux analystes d'obtenir rapidement une vision globale de l'état du système sans avoir recours à l'environnement immersif.

Les données sont présentées sous différentes formes :

- Graphes de relations entre entités.
- Statistiques et indicateurs de performance.
- Visualisation des clusters détectés
- Historique des événements et des alertes.

Filtrage et exploration des requêtes

L'application permet à l'utilisateur de filtrer les données selon plusieurs critères afin de faciliter l'analyse :

- Filtrage par type d'événement.
- Filtrage par niveau de risque.
- Filtrage par cluster.
- Recherche d'entités spécifiques.
- Sélection dynamique des données à afficher.

Ces fonctionnalités améliorent la rapidité d'exploration et permettent de se concentrer sur les éléments les plus pertinents.

Détection et affichage des anomalies

Les résultats du moteur d'analyse sont regroupés sous forme de clusters d'anomalies.

Pour chaque cluster, l'application affiche :

- L'identifiant du cluster.
- Le niveau de criticité.
- Les entités concernées.
- Les indicateurs associés.
- Les recommandations d'analyse.

Cette représentation permet à l'analyste d'identifier rapidement les zones nécessitant une attention particulière.

Communication avec le Backend

L'application Web communique avec le serveur backend à travers plusieurs API REST.

Ces interfaces de programmation permettent :

- La récupération des données d'analyse.
- L'obtention des résultats de détection d'anomalies.
- La consultation des informations relatives aux clusters.
- La mise à jour des données affichées en temps réel.
- L'envoi d'actions déclenchées par l'utilisateur.

Exemple de flux de communication

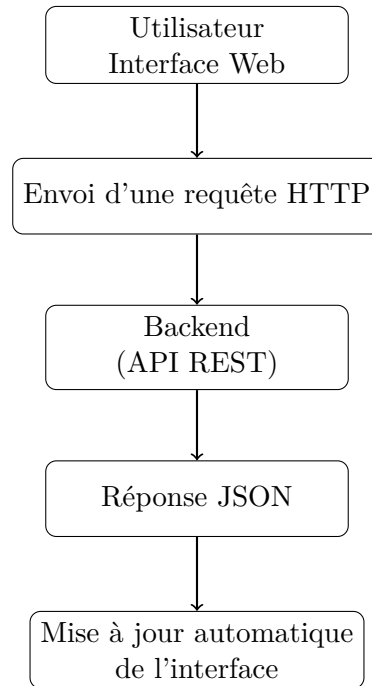


FIGURE 2 – Flux de communication entre l'application Web et le Backend

Interaction avec l'environnement VR

L'une des fonctionnalités majeures de l'application consiste à permettre à l'analyste d'interagir indirectement avec la scène de réalité virtuelle.

Les actions réalisées depuis l'interface Web peuvent être transmises au backend afin d'influencer l'environnement VR en temps réel. Par exemple :

- Sélection d'un cluster à analyser.
- Mise en évidence d'une anomalie spécifique.
- Changement du niveau de détail affiché dans la scène VR.

Technologies utilisées

Le développement de l'application Web s'appuie sur les technologies suivantes :

- Python
- Streamlit
- FastAPI
- API REST
- JSON

- Plotly pour les visualisations interactives

0.10.3 Interface utilisateur de l'application Web

Cette section présente les principales interfaces développées pour permettre à l'analyste d'explorer les données, visualiser les anomalies et interagir avec l'environnement VR.

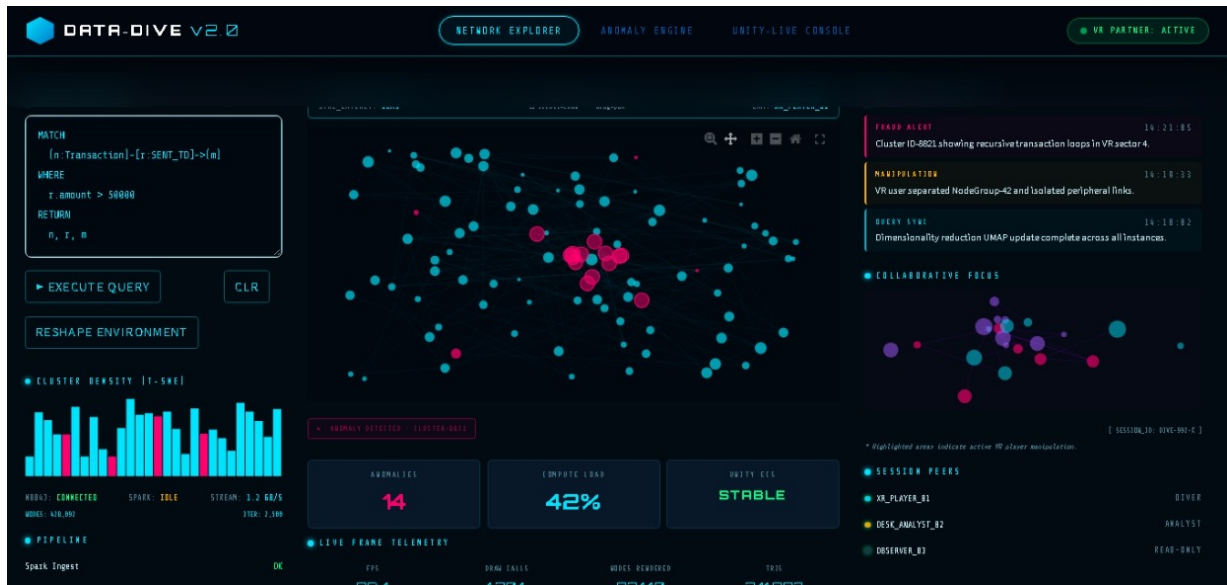


FIGURE 3 – Interface *Network Explorer* permettant la visualisation des relations entre les entités et l'exploration des clusters détectés.



FIGURE 4 – Interface *Anomaly Engine* présentant les anomalies détectées, leur niveau de criticité et les indicateurs associés.

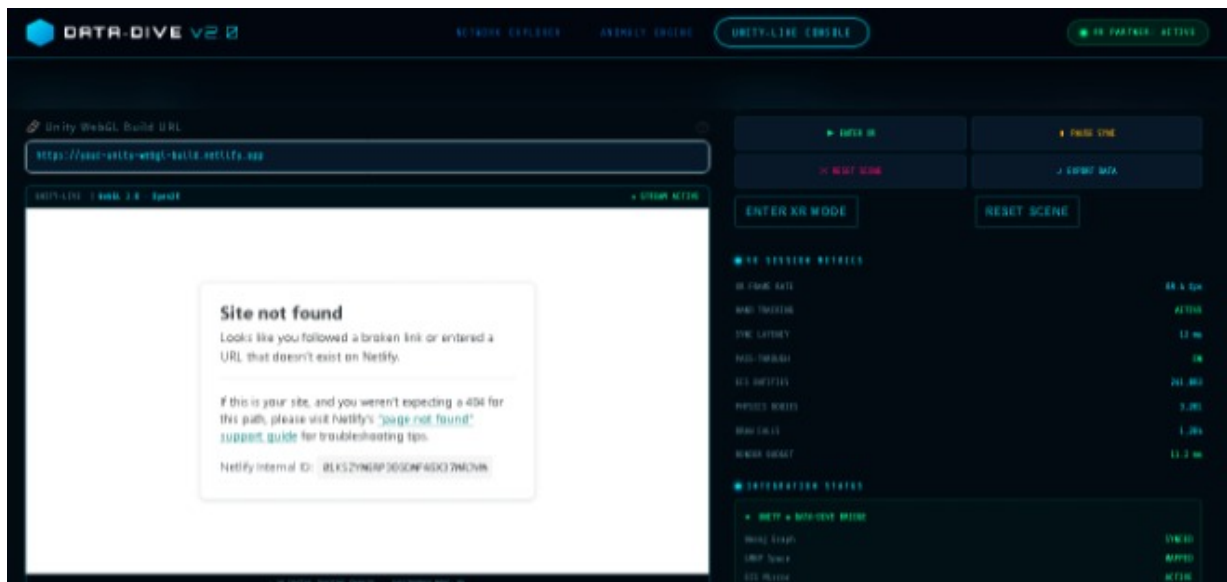


FIGURE 5 – Interface *Unity Live Console* assurant le suivi des échanges avec l’environnement VR et la communication avec les services backend.

0.11 Perspectives d’amélioration

Bien que le projet réponde aux objectifs fixés, plusieurs améliorations peuvent être envisagées afin d’enrichir ses fonctionnalités et d’améliorer l’expérience utilisateur. *Amélioration des analyses

- Intégration de modèles d’intelligence artificielle plus avancés pour améliorer la détection des anomalies.
- Mise en place d’algorithmes prédictifs permettant d’anticiper l’apparition de comportements suspects.
- Optimisation des performances pour le traitement de grands volumes de données.

Amélioration de l’application Web

- Ajout de nouveaux filtres et options de recherche avancée.
- Personnalisation des tableaux de bord selon les besoins des utilisateurs.
- Génération automatique de rapports d’analyse au format PDF ou Excel.
- Mise en place d’un système de notifications en temps réel pour les anomalies critiques.

0.11.1 Amélioration de l’environnement VR

- Ajout d’interactions plus immersives dans la scène de réalité virtuelle.
- Intégration de commandes vocales pour faciliter la navigation.
- Visualisation de données supplémentaires directement dans l’environnement VR.
- Support de nouveaux casques et équipements VR.

0.11.2 Amélioration de la communication et de l'architecture

- Migration vers une communication en temps réel (WebSocket) afin de réduire la latence entre les composants.
- Déploiement du système sur un serveur cloud pour un accès distant et multi-utilisateur.
- Renforcement de la sécurité des échanges entre les différents modules.

0.11.3 Travail collaboratif

- Gestion de plusieurs utilisateurs simultanés.
- Synchronisation en temps réel des actions des analystes.
- Ajout d'un système de commentaires et de partage des analyses.

Références

- PaySim Dataset – Kaggle : <https://www.kaggle.com/ntnu-testimon/paysim1>
- Documentation scikit-learn – Isolation Forest : <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- Documentation UMAP : <https://umap-learn.readthedocs.io/>
- Documentation NetworkX : <https://networkx.org/>